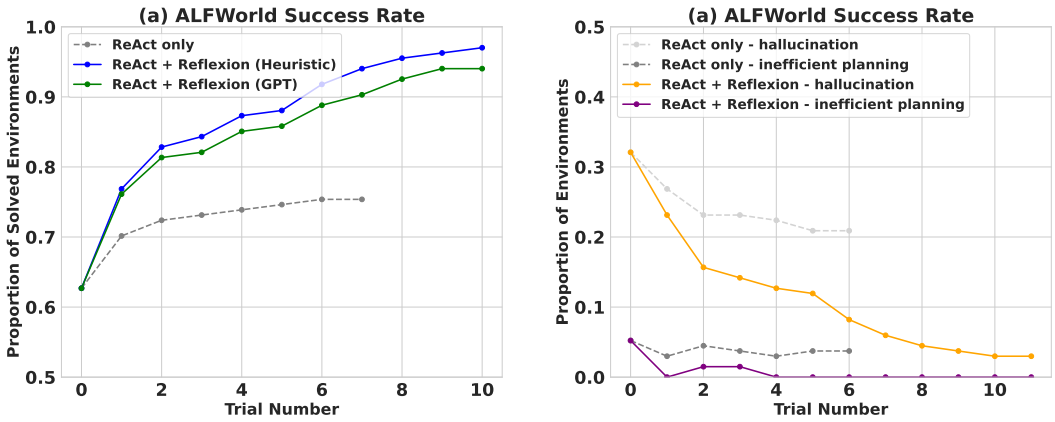

Figure 3: (a) AlfWorld performance across 134 tasks showing cumulative proportions of solved tasks using self-evaluation techniques of (Heuristic) and (GPT) for binary classification. (b) Classification of AlfWorld trajectories by reason of failure.

eliminates almost all of these cases by using self-reflection to distill long, failed trajectories into relevant experiences that can are used as "self-hints" in the future. There are two main cases in which long-term memory helps an agent in AlfWorld: 1) An early mistake in a long trajectory can be easily identified. The agent can suggest a new action choice or even a new long-term plan. 2) There are too many surfaces/containers to check for an item. The agent can exploit its experience memory over several trials to thoroughly search a room. In 3, the learning curve suggests that the learning process occurs over several experiences, meaning that the agent is successfully balancing cases 1 and 2 shown in the immediate spike in the improvement between the first two trials, then a steady increase over the next 11 trials to a near-perfect performance. On the other hand, 3 shows a ReAct-only agent converging at a hallucination rate of 22% with no signs of long-term recovery.

4.2 Reasoning: HotpotQA

HotPotQA [28] is a Wikipedia-based dataset with 113k question-and-answer pairs that challenge agents to parse content and reason over several supporting documents. To test improvement in reasoning *only* ability, we implement Reflexion + Chain-of-Thought (CoT) [26] for step-by-step $Q \rightarrow A$ and $Q, C_{gt} \rightarrow A$ implementations, where Q is the question, C_{gt} is the ground truth context from the dataset, and A is the final answer. Since CoT is not a multi-step decision-making technique, we give C_{gt} to the agent so that we can isolate the reasoning behavior over large sections of the provided text. To test holistic question and answering ability, which requires reasoning and action choice, we implement a Reflexion + ReAct [30] agent that can retrieve relevant context using a Wikipedia API and infer answers using step-by-step explicit thinking. For CoT implementations, we use 6-shot prompting; for ReAct, we use 2-shot prompting, and for self-reflection, we use 2-shot prompting. All examples can be found in the appendix.

Robustly evaluating natural language answers is a long-standing problem in NLP. Therefore, between trials, we use exact match answer grading using the environment to give a binary success signal to the agent. After each trial, the self-reflection loop is employed to amplify the binary signal, similar to the decision-making setup 4.1 in AlfWorld with a memory size of 3 experiences.

Results Reflexion outperforms all baseline approaches by significant margins over several learning steps. Furthermore, ReAct-only, CoT-only, and CoT (GT)-only implementations fail to probabilistically improve on any tasks, meaning that no failed tasks from the first trial from any of the baseline approaches were able to be solved in subsequent trials using a temperature of 0.7 In the Reflexion runs, we allowed the agent to gather experience and retry on failed tasks until it produced 3 consecutive failed attempts on the particular task. Naturally, the CoT (GT) achieved higher accuracy scores as it was given access to the ground truth context of the question. Still, the CoT (GT) agent is unable to correctly infer the correct answer for 39% of the questions, but Reflexion helps the agent to correct its mistakes without access to the ground truth answer to improve its accuracy by 14%.

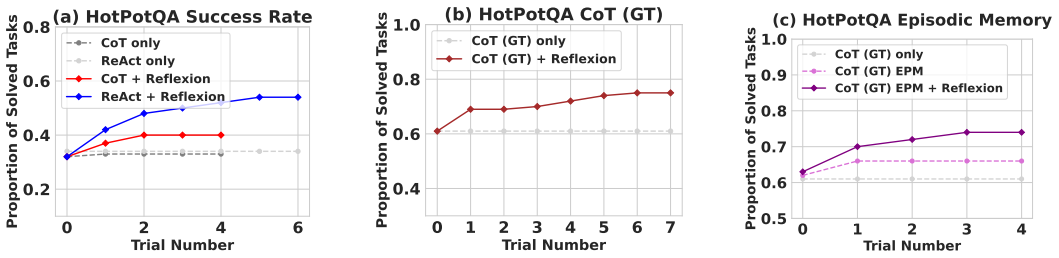

Figure 4: Chain-of-Thought (CoT) and ReAct. Reflexion improves search, information retrieval, and reasoning capabilities on 100 HotPotQA questions. (a) Reflexion ReAct vs Reflexion CoT (b) Reflexion CoT (GT) for reasoning only (c) Reflexion vs episodic memory ablation.

Analysis We perform an ablation experiment to isolate the advantage of the self-reflective step for reasoning using CoT (GT) as the baseline approach 4. Recall that CoT (GT) uses Chain-of-Thought reasoning with provided ground truth context, which tests reasoning ability over long contexts. Next, we add an element of episodic memory (EPM) by including the most recent trajectory. For the Reflexion agent, we implement the standard self-reflection step as a final pass. Intuitively, we test if the agent is iteratively learning more effectively by using verbal explanation using language written in the first person. 4 shows that self-reflection improves learning by an 8% absolute boost over the episodic memory learning advantage. This result supports the argument that refinement-only approaches are not as effective as self-reflection-guided refinement approaches.

4.3 Programming

We evaluate the baseline and Reflexion approaches on Python and Rust code writing on MBPP [2], HumanEval [6], and LeetcodeHardGym, our new dataset. MBPP and HumanEval measure function body generation accuracy given natural language descriptions. We use a benchmark language compiler, MultiPL-E [4], to translate subsets of HumanEval and MBPP to the Rust language. MultiPL-E is a collection of small compilers that can be used to translate Python benchmark questions to 18 other languages. We include experiments for Rust code generation to demonstrate that Reflexion implementations for code generation are language-agnostic and can be used for interpreted and compiled languages. Lastly, we introduce a new benchmark, LeetcodeHardGym, which is an interactive programming gym that contains 40 Leetcode hard-rated questions that have been released after October 8, 2022, which is the pre-training cutoff date of GPT-4 [18].

The task of programming presents a unique opportunity to use more grounded self-evaluation practices such as self-generated unit test suites. Thus, our Reflexion-based programming task implementation is eligible for pass@1 accuracy reporting. To generate a test suite, we use Chain-of-Thought prompting [26] to produce diverse, extensive tests with corresponding natural language descriptions. Then, we filter for syntactically valid test statements by attempting to construct a valid abstract syntax tree (AST) for each proposed test. Finally, we sample n tests from the collection of generated unit tests to produce a test suite T, denoted as $\{t_0, t_1, \ldots, t_n\}$. We set n to a maximum of 6 unit tests. Aside from the unit test suite component, the setup for the learning loop for a Reflexion programming agent is identical to the reasoning and decision-making agents with a max memory limit of 1 experience.

Benchmark + Language	Prev SOTA Pass@1	SOTA Pass@1	Reflexion Pass@1
HumanEval (PY)	65.8 (CodeT [5] + GPT-3.5)	80.1 (GPT-4)	**91.0**
HumanEval (RS)	–	60.0 (GPT-4)	**68.0**
MBPP (PY)	67.7 (CodeT [5] + Codex [6])	**80.1** (GPT-4)	77.1
MBPP (RS)	–	70.9 (GPT-4)	**75.4**
Leetcode Hard (PY)	–	7.5 (GPT-4)	**15.0**

Table 1: Pass@1 accuracy for various model-strategy-language combinations. The base strategy is a single code generation sample. All instruction-based models follow zero-shot code generation.

Benchmark + Language	Base	Reflexion	TP	FN	FP	TN
HumanEval (PY)	0.80	**0.91**	0.99	0.40	0.01	0.60
MBPP (PY)	**0.80**	0.77	0.84	0.59	0.16	0.41
HumanEval (RS)	0.60	**0.68**	0.87	0.37	0.13	0.63
MBPP (RS)	0.71	**0.75**	0.84	0.51	0.16	0.49

Table 2: Overall accuracy and test generation performance for HumanEval and MBPP. For Rust, HumanEval is the hardest 50 problems from HumanEval Python translated to Rust with MultiPL-E [4]. TP: unit tests pass, solution pass; FN: unit tests fail, solution pass; FP: unit tests pass, solution fail; TN: unit tests fail, solution fail.

Results Reflexion outperforms all baseline accuracies and sets new state-of-the-art standards on all benchmarks for Python and Rust except for MBPP Python 1. We further investigate the inferior performance of Reflexion on MBPP Python.

Analysis We acknowledge that self-reflecting code-generation agents are bound to their ability to write diverse, comprehensive tests. Therefore, in the case in which the model generates a flaky test suite, it is possible that all tests pass on an incorrect solution and lead to a false positive label on a code completion [11]. On the other hand, if the model produces an incorrectly written test suite, it is possible for some of the tests to fail on a correct solution, leading to a self-reflection generation that is conditioned on a false negative code completion. Given the implementation of Reflexion, false negatives are preferred over false positives as the agent may be able to use self-reflection to identify the incorrect test(s) and prompt itself to keep the original code completion intact. On the other hand, if an invalid test suite returns a false positive completion (all internal test cases pass but the implementation is incorrect), the agent will prematurely report an invalid submission. In 2, various conditions are measured to analyze performance beyond pass@1 accuracy. Previously, we displayed the inferior performance of Reflexion to the baseline GPT-4 on MBPP Python. In 2, we observe a notable discrepancy between the false positive labels produced by internal test execution, P(not pass@1 generation correct | tests pass). That is, the probability that a submission will fail given that it passes all unit tests. For HumanEval and MBPP Python, the baseline pass@1 accuracies are relatively similar, 82% and 80%, respectively. However, the false positive test execution rate for MBPP Python is 16.3% while the rate for HumanEval Python is a mere 1.4%, leading to 91% overall accuracy 1.

Approach	Test Generation	Self-reflection	Pass@1 (Acc)
Base model	False	False	0.60
Test generation omission	False	True	0.52
Self-reflection omission	True	False	0.60
Reflexion	True	True	**0.68**

Table 3: Pass@1 accuracy for various compromised approaches on the Reflexion approach using GPT-4 as the base model on HumanEval Rust - 50 hardest problems

Ablation study We test the composite approach of Reflexion for test generation and self-reflection cooperation on a subset of the 50 hardest HumanEval Rust problems. Our Rust compiler environment provides verbose error logs and helpful debugging hints, therefore serving as a good playground for compromised approaches. First, we omit internal test generation and execution steps, which test the agent to self-reflect without guidance from current implementations. 3 shows an inferior 52% vs 60% (baseline) accuracy, which suggests that the agent is unable to determine if the current implementation is correct without unit tests. Therefore, the agent must participate in all iterations of the run without the option to return early, performing harmful edits to the implementation.

Next, we test self-reflection contribution by omitting the natural language explanation step following failed unit test suite evaluations. Intuitively, this challenges the agent to combine the tasks of error identification and implementation improvement across all failed unit tests. Interestingly, the compromised agent does not improve performance over the baseline run. We observe that the test generation and code compilation steps are able to catch syntax and logic errors, but the implementation fixes do not reflect these indications. These empirical results suggest that several recent works that

propose *blind* trial and error debugging techniques without self-reflection are ineffective on harder tasks such as writing complex programs in Rust.

5 Limitations

At its core, Reflexion is an optimization technique that uses natural language to do policy optimization. Policy optimization is a powerful approach to improve action choice through experience, but it may still succumb to non-optimal local minima solutions. In this study, we limit long-term memory to a sliding window with maximum capacity, but we encourage future work to extend the memory component of *Reflexion* with more advanced structures such as vector embedding databases or traditional SQL databases. Specific to code generation, there are many practical limitations to test-driven development in specifying accurate input-output mappings such as non-deterministic generator functions, impure functions that interact with APIs, functions that vary output according to hardware specifications, or functions that invoke parallel or concurrent behavior that may be difficult to predict.

6 Broader impact

Large language models are increasingly used to interact with external environments (e.g. the Internet, software, robotics, etc.) and humans. Our work has the potential of reinforcing and empowering these agents toward greater automation and work efficiency, but it also amplifies the risks when these agents were put into misuse. We believe that this direction of research will need more effort in safety and ethical considerations.

On the other hand, reinforcement learning has suffered from its black-box policy and optimization setups in which interpretability and alignment have been challenging. Our proposed "verbal" reinforcement learning might address some of the issues and turn autonomous agents more interpretable and diagnosable. For example, in the case of tool-usage that may be too hard for humans to understand, self-reflections could be monitored to ensure proper intent before using the tool.

7 Conclusion

In this work, we present *Reflexion*, an approach that leverages verbal reinforcement to teach agents to learn from past mistakes. We empirically show that Reflexion agents significantly outperform currently widely-used decision-making approaches by utilizing self-reflection. In future work, Reflexion could be used to employ more advanced techniques that have been thoroughly studied in traditional RL settings, such as value learning in natural language or off-policy exploration techniques.

8 Reproducibility

We highly advise others to use isolated execution environments when running autonomous code writing experiments as the generated code is not validated before execution.

References

[1] Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., et al. (2022). Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*.

[2] Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. (2021). Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

[3] Brooks, E., Walls, L., Lewis, R. L., and Singh, S. (2022). In-context policy iteration. *arXiv preprint arXiv:2210.03821*.

[4] Cassano, F., Gouwar, J., Nguyen, D., Nguyen, S., Phipps-Costin, L., Pinckney, D., Yee, M.-H., Zi, Y., Anderson, C. J., Feldman, M. Q., Guha, A., Greenberg, M., and Jangda, A. (2022). Multipl-e: A scalable and extensible approach to benchmarking neural code generation.

[5] Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G., and Chen, W. (2022). Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*.

[6] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

[7] Chen, X., Lin, M., Schärli, N., and Zhou, D. (2023). Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*.

[8] Côté, M.-A., Kádár, A., Yuan, X., Kybartas, B., Barnes, T., Fine, E., Moore, J., Hausknecht, M., El Asri, L., Adada, M., et al. (2019). Textworld: A learning environment for text-based games. In *Computer Games: 7th Workshop, CGW 2018, Held in Conjunction with the 27th International Conference on Artificial Intelligence, IJCAI 2018, Stockholm, Sweden, July 13, 2018, Revised Selected Papers 7*, pages 41–75. Springer.

[9] Goodman, N. (2023). Meta-prompt: A simple self-improving language agent. *noahgoodman.substack.com*.

[10] Kim, G., Baldi, P., and McAleer, S. (2023). Language models can solve computer tasks. *arXiv preprint arXiv:2303.17491*.

[11] Lam, W., Winter, S., Wei, A., Xie, T., Marinov, D., and Bell, J. (2020). A large-scale longitudinal study of flaky tests. *Proc. ACM Program. Lang.*, 4(OOPSLA).

[12] Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. C. H. (2022). Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems*, 35:21314–21328.

[13] Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al. (2023). Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*.

[14] Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. (2022). Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097.

[15] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al. (2023). Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*.

[16] Nair, V., Schumacher, E., Tso, G., and Kannan, A. (2023). Dera: Enhancing large language model completions with dialog-enabled resolving agents. *arXiv preprint arXiv:2303.17071*.

[17] Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al. (2021). Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*.

[18] OpenAI (2023). Gpt-4 technical report. *ArXiv*.